

Aula 1 — Fundamentos da Qualidade

"Antes de automatizar: você sabe o que testar?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · Engenharia de Qualidade e Testes de Software · 24/09/2026

Bem-vindo(a) à disciplina!

Disciplina totalmente online **ao vivo**, 6 encontros síncronos de 3h30 (quintas, 19h-22h30) + trabalho assíncrono guiado entre aulas.

Estrutura:

- 6 aulas × 3h30 cada = 21h ao vivo + 3h assíncrono guiado = 24h totais
- Leituras dirigidas e labs entre um encontro e outro
- Office hours assíncrono via e-mail e fórum Canvas

| Bom momento pra começar: agora.

Agenda da aula (3h30)

- **90min** — Fundamentos: crise da qualidade, ISO/IEC 25010, tipos de teste, Pirâmide/Trophy/Honeycomb, exploratória (FEW HICCUPPS/SBTM)
- **15min** — Intervalo
- **90min** — Lab hands-on em sala: testes manuais no app TestesQAMobile, achando bugs reais por categoria
- **15min** — Wrap-up + preview Aula 2

Objetivos da Aula 1

- **Compreender** ISO/IEC 25010 (8 atributos de qualidade)
- **Classificar** tipos de teste (funcional, usabilidade, performance, etc.) e aplicar num app real
- **Diferenciar** Pirâmide de Cohn, Testing Trophy (Dodds), Honeycomb (Spotify)
- **Distinguir** web testing vs mobile testing
- **Praticar** teste exploratório com heurísticas (FEW HICCUPPS, charters)
- **Documentar** um bug encontrado num formato profissional

Quiz inicial — você concorda?

| *"Se o app não trava, ele já está bem testado."*

| *"Teste manual é só ficar clicando sem método."*

| *"Emulador testa igual a um device físico."*

| *"Achei poucos bugs = o app está pronto."*

| *"Só quem programa consegue testar de verdade."*

Spoiler: todas falsas. Veremos por quê — e vocês vão provar isso sozinhos no lab de hoje.

A crise da qualidade — mobile

- **90%** dos usuários já pararam de usar um app por causa de mau desempenho (crash, travamento, lentidão) — [pesquisa citada pela APMdigest](#)
- Fragmentação agrava tudo: **milhares** de combinações OS + versão + fabricante só no Android
- 1 review de 1 estrela por bug crítico é visível **pra sempre** na loja — diferente de um bug corrigido silenciosamente na web

| Testar não é burocracia. É a diferença entre o app sobreviver na loja ou não.

A crise da qualidade — web

- **53%** dos visitantes mobile abandonam um site que demora **mais de 3s** pra carregar — [Google, The Need for Mobile Speed](#)
- Web soma sua própria fragmentação: **browsers** (Chromium/Firefox/WebKit) × **viewports** × **JS habilitado ou não**
- Bugs de software (mobile + web) custam **US\$ 59,5 bilhões/ano** à economia dos EUA, mais da metade recai sobre o **usuário final** — [Tasse, G. \(2002\), NIST Planning Report 02-3](#)

Na web, o concorrente está **1 clique** de distância — performance ruim não dá segunda chance.

ISO/IEC 25010:2023 — qualidade de produto

8 atributos:

1. **Functional Suitability** — completude funcional
2. **Performance Efficiency** — eficiência de recursos
3. **Compatibility** — interoperabilidade
4. **Usability** — facilidade de uso
5. **Reliability** — disponibilidade, robustez
6. **Security** — proteção contra ameaças
7. **Maintainability** — facilidade de mudança
8. **Portability** — adaptabilidade entre ambientes

| Norma é vocabulário comum. Use em Test Strategy Documents.

Tipos de teste — a classificação que vamos usar hoje

Categoria	Pergunta que responde
Funcional	O recurso faz o que deveria?
Usabilidade	É fácil e intuitivo de usar?
Interface (UI)	Layout, alinhamento, responsividade estão corretos?
Performance	Responde rápido? Consome poucos recursos?
Compatibilidade	Funciona em diferentes OS/devices/versões?
Segurança	Resiste a uso malicioso ou dados sensíveis expostos?
Acessibilidade	Pessoas com deficiência conseguem usar?
Instalação/Conectividade	Instala certo? Se comporta bem offline/rede ruim?

Cada bug tem uma categoria. Categorizar é o primeiro passo pra saber **onde** olhar e **o que** priorizar.

Como testar cada tipo?

- **Com roteiro (scripted):** casos de teste pré-definidos, passo a passo, resultado esperado conhecido. Bom pra regressão e cobertura previsível.
- **Sem roteiro (exploratório):** você define o objetivo, mas descobre os passos testando ao vivo — usa **heurísticas** pra guiar em vez de um script fixo.

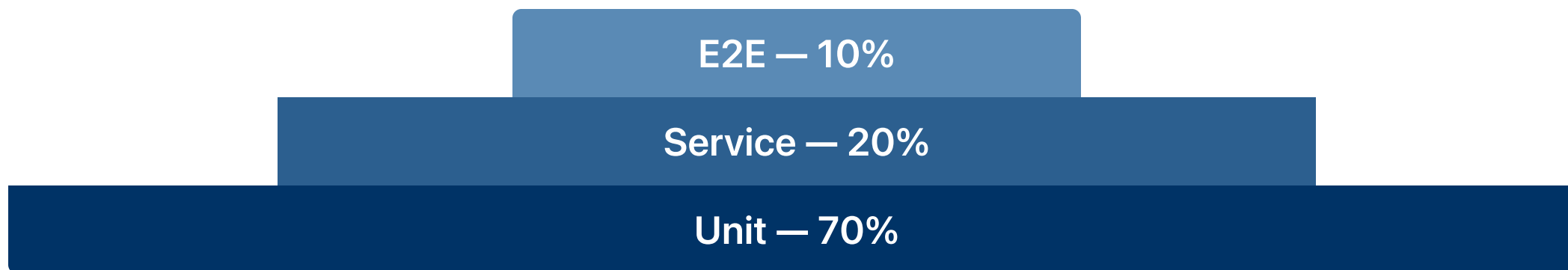
Heurística FEW HICCUPPS (James Bach) — pergunte, comparando o produto com:

História · Imagem · Comparáveis · Claims · Usuários · Produto · Propósito · Standards

Charter (SBTM — Session-Based Test Management): missão curta e cronometrada — *"Explorar [feature] procurando problemas de [tipo], usando [dados/condições]"*.

Hoje: 2 exercícios **com roteiro** (fornecido no app) — a base antes de ir pro exploratório puro nas próximas aulas.

Pirâmide de testes (Cohn) — clássica web

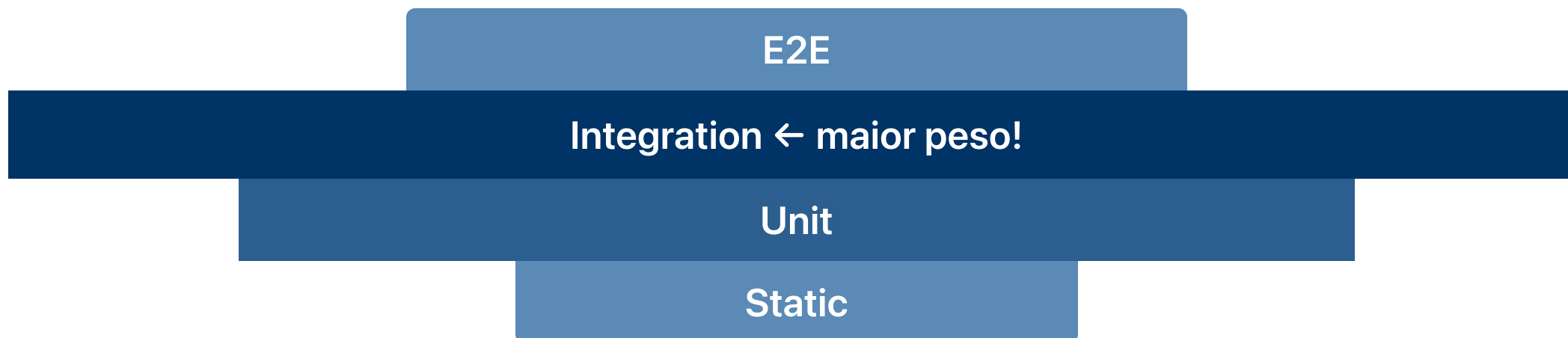


Por que pirâmide? E2E é caro, lento, frágil. Unit é barato, rápido, focado.

Aplica-se bem em projetos com bom design (interfaces estáveis, dependências claras).

Testing Trophy — Kent C. Dodds

Alternativa para apps JavaScript modernos:



Argumento: integration tests dão mais confidence per dollar em apps com muitos mocks de unit.

— Não é "pirâmide errada" — é trade-off contextual.

Honeycomb — Spotify

Para microsserviços:

Integrated — poucos mas vitais

Integration ← grande peso

Implementation ← grande peso (= unit)

| Spotify defende que microsserviços precisam de balanço unit + integration; não de muitos E2E.

Quando usar qual?

Estratégia	Bom para
Pirâmide Cohn	Sistemas com interfaces estáveis, equipes pequenas
Testing Trophy	Apps web/RN com data fetching + state management
Honeycomb	Microserviços, equipes distribuídas

Não há resposta universal. **Contexto define.**

Web testing vs Mobile testing

Aspecto	Web	Mobile
Plataformas	Chromium, Firefox, WebKit	iOS + Android (+ minoritárias)
Atualização	Refresh	Build + store
Fragmentação	Browsers + viewports	OS + devices + sensores
Sensores	Limitado	Câmera, GPS, biometria, NFC
Permissões	Cookies/storage	+ camera, location, contacts, push
Background	Limitado	Lifecycle complexo (foreground/background)

Tempo desta disciplina: **1 aula fundamentos + 2 aulas web + 1 aula mobile + 1 aula IA + 1 aula CI/CD e Projeto Final** (6 aulas no total).

Automação existe — mas não é hoje

POM (Page Object Model) e **Screenplay Pattern** são padrões pra estruturar testes automatizados (Playwright/Selenium). **Flakiness** (teste que passa/falha aleatoriamente, sem mudar o código) é o problema nº1 de quem automatiza.

Isso é conteúdo da **Aula 2 (Playwright Avançado)** — com código de verdade. Hoje ainda não escrevemos automação: primeiro aprendemos **o que** testar manualmente, pra depois saber **o que vale a pena** automatizar.

Regra prática: só automatize um teste que você já executou manualmente e entende exatamente o que ele verifica.

Onde rodar o teste — device, emulador, farm

Ambiente	Prós	Contras
Device físico	Comportamento real (sensores, performance, rede)	Você só tem 1-2 modelos
Emulador/Simulador	Grátis, rápido de resetar, várias versões de OS	Não pega bugs de hardware real (câmera, GPS real)
Cloud device farm (BrowserStack, Firebase Test Lab)	Matriz real de devices/OS sem comprar todos	Custo, tem free tier limitado

O app de hoje: testem no **seu próprio celular** (device físico) — é o ambiente mais realista que vocês têm à mão agora.

Como documentar um bug (formato profissional)

Campo	Exemplo
ID	BUG-001
Tipo	Funcional
Pré-condição	App aberto, tela de Cadastro
Passos	1. Digitar "a@" no campo email · 2. Tocar em Salvar
Resultado esperado	Erro de validação, não salva
Resultado obtido	Salva normalmente, sem erro

Sem isso, um bug reportado vira "não funciona" — inútil pra quem vai corrigir. É o formato que vocês usam no Passo 3 do lab de hoje.

Test Strategy Document

Documento que comunica estratégia de teste:

1. **Escopo** — o que testa, o que não testa
2. **Tipos de teste** — pirâmide aplicada
3. **Ferramentas** — stack escolhida com justificativa
4. **Ambientes** — dev, staging, prod, devices
5. **Métricas** — flake rate, escape rate, coverage
6. **Responsabilidades** — quem escreve, quem mantém
7. **CI/CD** — gates, sharding

| Documento vivo, revisado a cada release major.

Lab hands-on (90min) — testes manuais por tipo

Sem código hoje. App real publicado (App Store/Play Store), bugs propositalmente organizados por categoria — aplica direto os "tipos de teste" que acabamos de ver.

Roteiro

1. Abrir o app TestesQAMobile (App Store / Play Store) (10min)
2. Exercício 1.1 — Cadastro de Usuário (Funcional): roteiro + casos já no app, achar 3 bugs (30min)
3. Exercício 2.1 — Onboarding (Usabilidade): mesma mecânica, achar 3 bugs (30min)
4. Documentar 1 bug de cada no formato ID/Tipo/Passos/Resultado (15min)
5. Fechamento: teste manual é qual fatia da pirâmide? (5min)

Roteiro: `exercicios/00-warmup-aula1-tipos-de-teste/` · App publicado, código privado — testamos de fora, como QA real.

Leituras obrigatórias

- Crispin, L.; Gregory, J. — *Holistic Testing* (Pearson 2024), cap 1–3
- Bolton, M. — *FEW HICCUPPS* (heurística de oráculo, usada no lab de hoje)
- Bach, J. — *Session-Based Test Management*
- Dodds, K. — *Write Tests. Not Too Many. Mostly Integration.*
- ISO/IEC 25010:2023 — resumo guiado (link no Canvas)
- Spotify Engineering — *Testing of Microservices* (Honeycomb)

Bibliografia completa da disciplina: [BIBLIOGRAFIA.md](#)

Próxima aula — Aula 2 (01/10)

Playwright Avançado — começamos a automatizar o que hoje foi manual

- Page Object Model + Screenplay Pattern
- Flakiness: métricas, detecção, mitigação
- Locators recomendados
- Network mocking + HAR replay
- storageState (auth reuse)
- Visual regression + sharding 4-way em CI

Bons estudos!

✉ jackson.96@gmail.com (também canal Canvas)

🔗 linkedin.com/in/3jacksonsmith

🐙 github.com/jacksonsmith

Office hours: quintas 19h via Google Meet (link Canvas)